TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HỒ CHÍ MINH

VIỆN CÔNG NGHỆ VIỆT - NHẬT

ĐỒ ÁN MÔN HỌC
HỌC PHẦN : BẢO MẬT THÔNG TIN

ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG XÁC THỰC KHÔNG MẬT KHẨU DỰA TRÊN TIÊU CHUẨN FIDO2 VÀ WEBAUTHN CHO ỨNG DỤNG WEB

Thành viên đề tài:	Trương Nhựt Huy	24800600403
	Nguyễn Trình Khoa	24800600488
	Nguyễn Quốc Kiệt	24800612252
	Ngô Thành Đạt	24800600203

Giảng viên hướng dẫn: Lê ThS. Nguyễn Trọng Minh Hồng Phước

TP. Hồ Chí Minh, 10/2026

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa, bảo mật thông tin và xác thực định danh trở thành một trong những thách thức cốt lõi của ngành Khoa học Máy tính. Nhiều thập kỷ qua, mật khẩu luôn là phương thức xác thực phổ biến nhất nhưng lại bộc lộ vô số điểm yếu chí mạng. Giới hạn trí nhớ khiến người dùng thường xuyên đặt mật khẩu yếu hoặc tái sử dụng chúng, tạo kẽ hở cho các cuộc tấn công lừa đảo phi kỹ thuật (phishing), nhồi thông tin xác thực hay rò rỉ dữ liệu khiến hàng tỷ tài khoản bị xâm nhập trái phép.

Để giải quyết triệt để vấn đề này, Liên minh FIDO (Fast IDentity Online) và Tổ chức W3C (World Wide Web Consortium) đã phối hợp ban hành tiêu chuẩn xác thực WebAuthn (Web Authentication) thuộc bộ tiêu chuẩn FIDO2. Giao thức mang tính cách mạng này cho phép đăng nhập an toàn tuyệt đối thông qua sinh trắc học được tích hợp sẵn trên thiết bị (Windows Hello, Touch ID, Face ID...) hoặc khóa bảo mật ngoại vi mà không cần ghi nhớ hay nhập mật khẩu. Hoạt động dựa trên nền tảng mã hóa khóa công khai (Public Key Cryptography), FIDO2 mang lại khả năng chống lừa đảo mạng ở mức độ cao nhất hiện nay.

Nhận thấy tính cấp thiết và xu hướng tất yếu của công nghệ xác thực không mật khẩu (Passwordless), nhóm chúng em quyết định chọn đề tài **"Nghiên cứu và triển khai hệ thống xác thực không mật khẩu dựa trên tiêu chuẩn FIDO2/WebAuthn"** làm đồ án môn học. Bằng việc ứng dụng ngôn ngữ Node.js, đồ án tập trung bóc tách cơ chế hoạt động của WebAuthn, từ đó thiết kế và xây dựng hệ thống mô phỏng quy trình Đăng ký và Đăng nhập bằng sinh trắc học. Thông qua đồ án này, nhóm mong muốn làm chủ quy trình triển khai Backend và đóng góp một tài liệu nghiên cứu chi tiết về một trong những chuẩn bảo mật hiện đại nhất thế giới.

LỜI CẢM ƠN

Để có thể hoàn thành đồ án môn học này một cách chỉn chu và trọn vẹn nhất, chúng em xin gửi lời cảm ơn chân thành đến **Ban Giám hiệu Trường Đại học HUTECH và Viện Công nghệ Việt - Nhật (VJIT)** đã tạo điều kiện học tập tốt nhất, cung cấp môi trường nghiên cứu năng động, trang bị thiết bị thực hành

và xây dựng cho chúng em những nền tảng kiến thức vô cùng vững chắc về lập trình cũng như về vấn đề bảo mật trong công nghệ thông tin.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến **Giảng viên hướng dẫn – ThS. Nguyễn Trọng Minh Hồng Phước**. Nhờ những bài giảng tận tâm, sự hướng dẫn chi tiết và những định hướng đúng đắn của Thầy, chúng em mới có thể vượt qua các khó khăn kỹ thuật để hoàn thiện đồ án "Nghiên cứu và triển khai hệ thống xác thực không mật khẩu dựa trên tiêu chuẩn FIDO2/WebAuthn".

Do giới hạn về mặt thời gian nghiên cứu cũng như kinh nghiệm triển khai thực tiễn hệ thống lớn còn nhiều hạn chế, đồ án chắc chắn không thể tránh khỏi những thiếu sót nhất định về mặt tối ưu hóa hay quy mô cơ sở dữ liệu. Chúng em rất mong nhận được sự thông cảm, những lời nhận xét đánh giá và góp ý quý báu từ Thầy để nhóm có thể rút kinh nghiệm, trau dồi thêm kỹ năng và tiếp tục hoàn thiện.

Chúng em xin chân thành cảm ơn!

NHIỆM VỤ ĐỒ ÁN

1. **Nghiên cứu lý thuyết:** Tìm hiểu mã hóa khóa công khai, kiến trúc tiêu chuẩn FIDO2 và giao thức WebAuthn (W3C) trong ứng dụng chống lừa đảo.

2. Thiết kế luồng hoạt động (Data Flow): Xây dựng sơ đồ quy trình chuẩn cho hai giai đoạn cốt lõi: Đăng ký (Registration) và Đăng nhập (Authentication) bằng Passkey.

3. Xây dựng Backend (Node.js): Tích hợp @simplewebauthn/server để tạo "Thử thách" (Challenge), xác minh chữ ký số, lưu Khóa công khai và cập nhật bộ đếm chống tấn công phát lại.

4. Tích hợp Frontend UI: Lập trình JavaScript gọi API WebAuthn để kích hoạt cảm biến sinh trắc học (Windows Hello, Touch ID) nhằm tạo và sử dụng Passkey.

5. Tối ưu hóa cơ chế Xác minh (UV): Cấu hình userVerification: 'preferred' nhằm ưu tiên xác minh chính chủ bằng sinh trắc học (UV) thay vì chỉ kiểm tra sự hiện diện vật lý (UP).

6 .Kiểm thử thực nghiệm: Vận hành hệ thống trên localhost, sử dụng Live Security Inspector theo dõi luồng dữ liệu trao đổi để chứng minh độ an toàn của giao thức.

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN HỆ THỐNG VÀ VẤN ĐỀ NGHIÊN CỨU

1.1. Tổng quan về vấn đề được nghiên cứu

Xác thực người dùng là module không thể thiếu trong các ứng dụng web hiện nay. Dù phương pháp dùng mật khẩu kết hợp mã OTP (Xác thực hai yếu tố - 2FA) rất phổ biến, nhưng nó vẫn bộc lộ nhiều rủi ro. Người dùng thường đặt mật khẩu yếu hoặc dùng chung cho nhiều trang, trong khi tin tặc có thể dùng kỹ thuật

Commented [KNQ1]:

lừa đảo (Phishing) tạo các trang web giả mạo để lừa người dùng nhập và chiếm đoạt cả mật khẩu lẫn OTP.

Để khắc phục triệt để lỗ hổng này, chuẩn xác thực FIDO2/WebAuthn đã ra đời. Giao thức này cho phép đăng nhập bằng sinh trắc học (vân tay, khuôn mặt) hoặc mã PIN phần cứng, hoàn toàn loại bỏ việc sử dụng mật khẩu. Dựa trên cơ chế mã hóa khóa công khai (Public Key Cryptography), FIDO2 đảm bảo khóa bảo mật không thể bị đánh cắp hay làm giả.

Xuất phát từ tính cấp thiết đó, đồ án này được thực hiện nhằm nghiên cứu sâu về cơ chế hoạt động của WebAuthn. Trọng tâm của đồ án là xây dựng một hệ thống Backend bằng Node.js kết hợp với giao diện Frontend, qua đó mô phỏng hoàn chỉnh quy trình tạo Passkey (khóa bảo mật) và xác thực đăng nhập không mật khẩu trên môi trường thực tế

1.2. Khảo sát thực trạng về an toàn thông tin

Hiện nay, một người dùng trung bình có rất nhiều tài khoản trực tuyến từ mạng xã hội, email, đến tài khoản ngân hàng. Việc bắt người dùng nhớ các mật khẩu phức tạp, dài và khác nhau cho từng trang web là không khả thi. Điều này dẫn đến tình trạng rò rỉ dữ liệu (Data Breach) rất phổ biến. Khi một trang web bị hack, hacker sẽ lấy được danh sách email và mật khẩu, sau đó dùng tool tự động để thử đăng nhập vào các trang web khác (tấn công Credential Stuffing - Tấn công nhồi thông tin xác thực).

Để đối phó với tình trạng này, các công ty công nghệ lớn như Google, Apple, Microsoft đã liên minh lại (FIDO Alliance) để đẩy mạnh việc sử dụng "Passkey" thay cho mật khẩu. Hiện tại, các trình duyệt phổ biến (Chrome, Edge, Safari) và các hệ điều hành (Windows 11, iOS, Android) đều đã hỗ trợ API WebAuthn. Vì vậy, việc sinh viên IT tìm hiểu và biết cách tích hợp xác thực FIDO2 vào ứng dụng web là một kỹ năng rất cần thiết, bắt kịp với xu hướng công nghệ hiện đại.

Commented [KNQ2]: Ghi rõ passkey là gì hihi

1.3. Đánh giá về giao thức xác thực FIDO2/WebAuthn

Dựa trên quá trình tìm hiểu lý thuyết, nhóm đánh giá giao thức FIDO2/WebAuthn có những ưu nhược điểm như sau:

- Ưu điểm:

Chống lừa đảo (Phishing-resistant) hiệu quả: Khi tạo Passkey, khóa này sẽ được gắn chặt với tên miền của trang web (Origin). Nếu người dùng bị lừa vào trang web giả mạo (ví dụ trang thật là facebook.com

nhưng trang giả là faceb00k.com), trình duyệt sẽ nhận ra sự khác biệt tên miền và không cho phép gửi chữ ký bảo mật lên. Hacker không thể lấy cắp được thông tin đăng nhập.

Tiện lợi cho người dùng: Khách hàng không cần phải nhớ hay gõ mật khẩu. Chỉ cần quét vân tay hoặc nhìn vào màn hình là đăng nhập thành công. Thời gian xử lý rất nhanh, cải thiện trải nghiệm người dùng (UX).

An toàn cho phía Server: Máy chủ (Server) chỉ lưu trữ Khóa công khai (Public Key) chứ không lưu mật khẩu. Khóa công khai này nếu bị hacker đánh cắp từ cơ sở dữ liệu cũng vô dụng vì không thể dùng nó để đăng nhập hoặc dịch ngược ra Khóa bí mật (Private Key) đang nằm trên máy người dùng.

- Nhược điểm:

Yêu cầu về phần cứng: Giao thức này bắt buộc thiết bị người dùng (điện thoại, laptop) phải có cảm biến sinh trắc học hoặc có cài đặt mã PIN bảo vệ máy (TPM). Nếu người dùng xài máy tính đời cũ không hỗ trợ, họ sẽ không thể sử dụng tính năng này.

Vấn đề khôi phục tài khoản (Recovery): Khóa bí mật nằm trên thiết bị vật lý của người dùng. Nếu người dùng làm mất điện thoại hoặc cài lại Win mà quên backup, họ sẽ không thể đăng nhập. Do đó, hệ thống thực tế cần phải code thêm các phương án dự phòng (như gửi link đăng nhập qua email) làm hệ thống phức tạp hơn.

1.4. Mục tiêu nghiên cứu

Đồ án được thực hiện nhằm đạt được hai mục tiêu chính sau:

Về mặt lý thuyết: Hiểu rõ bản chất của hệ mật mã khóa công khai và nắm vững luồng hoạt động của giao thức FIDO2/WebAuthn.

Về mặt thực tiễn: Xây dựng thành công một hệ thống mô phỏng (gồm Frontend và Backend Node.js) có khả năng giao tiếp với các thiết bị sinh trắc học của hệ điều hành (Windows Hello, Touch ID), từ đó thực hiện việc đăng ký và đăng nhập không cần mật khẩu một cách trơn tru, an toàn.

1.5. Đối tượng và phạm vi của đồ án ứng dụng

- Đối tượng: Đồ án được thực hiện nhằm mục đích phục vụ báo cáo môn học, giúp các thành viên trong nhóm hiểu rõ cách thức triển khai xác thực Passwordless vào một ứng dụng web thực tế.

- Phạm vi: Do thời gian và nguồn lực có hạn, hệ thống được triển khai và chạy thử nghiệm trên môi trường máy chủ cục bộ (localhost). Về phần cơ sở dữ liệu, nhóm sử dụng mảng dữ liệu lưu tạm trên RAM (trong

file db.js) thay vì dùng các hệ quản trị CSDL như MySQL hay MongoDB. Cách tiếp cận này giúp nhóm tập trung tối đa vào việc xử lý logic phức tạp của FIDO2 thay vì mất thời gian cấu hình Database, phù hợp với mục tiêu chứng minh khái niệm (Proof of Concept) của đồ án.

1.6. Cấu trúc nội dung đồ án

Báo cáo đồ án được chia thành 5 chương với nội dung như sau:

- Chương 1: Tổng quan hệ thống và vấn đề nghiên cứu. Giới thiệu lý do chọn đề tài, khảo sát thực trạng mật khẩu hiện nay, đánh giá ưu nhược điểm của FIDO2 và các nhiệm vụ cần làm.

- Chương 2: Cơ sở lý thuyết và công nghệ ứng dụng. Nêu các khái niệm lý thuyết về mã hóa bất đối xứng, cách hoạt động của WebAuthn và giới thiệu về Node.js, thư viện SimpleWebAuthn.

- Chương 3: Phân tích và Thiết kế hệ thống. Xây dựng sơ đồ Use Case, sơ đồ luồng dữ liệu (Sequence Diagram) và mô tả cấu trúc lưu trữ dữ liệu của hệ thống.

- Chương 4: Triển khai và Kết quả thực nghiệm. Trình bày một số đoạn code chính xử lý ở Backend, quá trình khắc phục các lỗi về xác thực phần cứng và hiển thị hình ảnh giao diện chạy thực tế của hệ thống.

- Chương 5: Kết luận và Hướng phát triển. Tổng kết những gì nhóm đã làm được, nêu ra các hạn chế (như việc lưu CSDL tạm thời) và định hướng phát triển ứng dụng lên các host thực tế trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ỨNG DỤNG

2.1. Nền tảng mật mã khóa công khai (Public-Key Cryptography) trong FIDO2

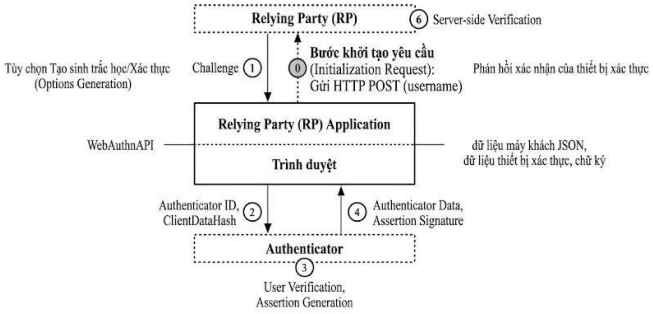
Trong các hệ thống xác thực truyền thống, hệ thống thường lưu trữ mật khẩu dưới dạng chuỗi văn bản thuần hoặc thông qua hàm băm (Hash). Mô hình này tạo ra rủi ro cốt lõi là máy chủ phải nắm giữ "bí mật chia sẻ" (Shared Secret). Khi cơ sở dữ liệu bị tấn công, toàn bộ tài khoản người dùng đứng trước nguy cơ bị chiếm đoạt.

Tiêu chuẩn FIDO2 loại bỏ hoàn toàn khái niệm mật khẩu tĩnh bằng cách áp dụng Hệ mật mã bất đối xứng (Asymmetric Cryptography). Mỗi khi người dùng đăng ký tài khoản với một dịch vụ trực tuyến (Relying Party - RP), thiết bị sẽ tự động khởi tạo một cặp khóa độc nhất vô nhị gắn liền với dịch vụ đó.

Khóa bí mật (Private Key): Được lưu trữ cố định và bảo vệ tuyệt đối bên trong phân vùng phần cứng an toàn của thiết bị (như vi mạch TPM trên máy tính hoặc Secure Enclave trên điện thoại). Khóa này mang thuộc tính "không thể trích xuất" (non-extractable), tuyệt đối không bao giờ rời khỏi thiết bị dưới bất kỳ hình thức nào.

Khóa công khai (Public Key): Được truyền tải và lưu trữ an toàn trên cơ sở dữ liệu của máy chủ ứng dụng. Khóa này chỉ có một nhiệm vụ duy nhất là giải mã và kiểm chứng chữ ký số do Private Key tương ứng tạo ra.

Nhờ nền tảng mã hóa này, ngay cả khi cơ sở dữ liệu của máy chủ bị rò rỉ hoàn toàn, kẻ tấn công chỉ thu được các Public Key vốn không có giá trị để đăng nhập hay tính ngược ra Private Key.



Hình 2.1: Sơ đồ minh họa nguyên lý mã hóa khóa công khai

Commented [KNQ3]: Em nghĩ nên nói rõ các option: 0, 1, 2, 3,...

Quy trình mã hóa bất đối xứng hoạt động theo nguyên tắc trói buộc toán học (Hình 2.1). Dữ liệu (chữ ký số) được tạo ra bởi Khóa bí mật (Private Key) chỉ có thể được kiểm chứng và giải mã thành công bởi duy nhất Khóa công khai (Public Key) cùng cặp với nó. Nguyên lý này giúp máy chủ hoàn toàn xác minh được danh tính của thiết bị gửi mà không cần phải yêu cầu truyền dữ liệu gốc qua mạng Internet.

2.2. Kiến trúc tổng thể và các thành phần cốt lõi của FIDO2

FIDO2 không đơn thuần là một giao thức độc lập, mà là một hệ sinh thái tiêu chuẩn mở do Liên minh FIDO (FIDO Alliance) và Tổ chức W3C đồng phát triển. Kiến trúc hệ thống bao gồm ba thực thể cốt lõi tương tác chặt chẽ với nhau:

2.2.1. Bên phụ thuộc (Relying Party - RP):

Là máy chủ ứng dụng (Web Server) cung cấp dịch vụ cho người dùng. Trong đồ án thực tế của nhóm, RP chính là phần mã nguồn Backend viết bằng Node.js (sử dụng framework Express). RP có trách nhiệm:

1. Sinh ra chuỗi dữ liệu ngẫu nhiên chống tấn công phát lại gọi là Thử thách (Challenge).
2. Quản lý chính sách xác thực và ràng buộc tên miền thông qua RP ID.
3. Lưu trữ Credential ID và Public Key của người dùng.

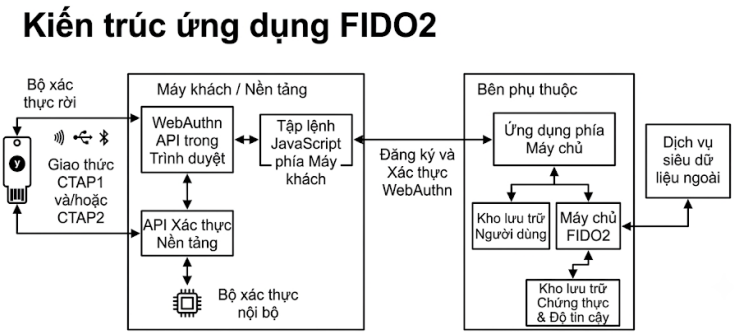
2.2.2. Máy khách (Client / User Agent):

Là trình duyệt web (Google Chrome, Microsoft Edge, Safari) hoặc hệ điều hành (Windows 11, iOS, Android). Máy khách đóng vai trò trung gian nhận lệnh từ RP thông qua chuẩn API web, đồng thời thực thi chính sách Ràng buộc tên miền (Domain Binding) nhằm ngăn chặn tuyệt đối các cuộc tấn công lừa đảo (Phishing).

2.2.3. Bộ xác thực (Authenticator):

Là thực thể phần cứng hoặc phần mềm an toàn thực thi các thuật toán mật mã và xác minh danh tính người dùng. Bộ xác thực được chia làm hai loại:

1. Platform Authenticator (Bộ xác thực nội bộ): Tích hợp sẵn trong thiết bị như Windows Hello (trên Windows) hoặc Touch ID / Face ID (trên thiết bị Apple).
2. Roaming Authenticator (Bộ xác thực rời): Các thiết bị phần cứng ngoại vi như khóa bảo mật USB YubiKey kết nối qua cổng USB, NFC hoặc Bluetooth.



Hình 2.2: Sơ đồ 3 thành phần của hệ thống WebAuthn/FIDO2

Toàn bộ hệ sinh thái FIDO2 vận hành dựa trên sự kết hợp chặt chẽ của hai luồng giao tiếp **(Hình 2.2)**. Trong khi WebAuthn API đảm nhận việc truyền tải dữ liệu an toàn qua mạng lưới Internet giữa Trình duyệt và Máy chủ (RP), thì giao thức

CTAP đóng vai trò cầu nối nội bộ, cho phép hệ điều hành tương tác trực tiếp với các vi mạch xác thực phần cứng. Sự phân tách kiến trúc này giúp hệ thống cô lập rủi ro bảo mật ở mức tối đa.

2.3. Các thông số bảo mật chuyên sâu trong giao thức:

Để hệ thống hoạt động an toàn và chống lại các kịch bản tấn công tinh vi, FIDO2 vận hành dựa trên các thông số bảo mật cốt lõi sau :

2.3.1. Thử thách (Challenge) và Chống tấn công phát lại (Replay Attack) :

Challenge là một chuỗi byte ngẫu nhiên do máy chủ RP tạo ra cho mỗi phiên đăng nhập hoặc đăng ký. Khi người dùng yêu cầu đăng nhập, thiết bị sẽ dùng Private Key ký lên chuỗi Challenge này. Nếu tin tặc bắt lén được gói tin chữ ký trên mạng và thử gửi lại vào lần sau (Replay Attack), máy chủ sẽ đối chiếu và từ chối ngay lập tức vì Challenge của phiên mới đã thay đổi hoàn toàn so với phiên cũ.

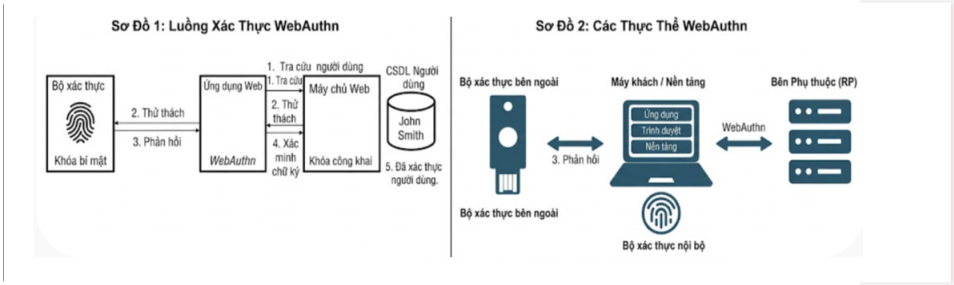
2.3.2. Định danh máy chủ (RP ID) và Tên miền nguồn gốc (Origin) :

Đây là hai thông số đóng vai trò then chốt, tạo nên cơ chế chống lừa đảo (Phishing-resistant) cốt lõi của giao thức FIDO2:

Origin: Trình duyệt tự động thu thập tên miền thực tế trên thanh địa chỉ (ví dụ: *https://example.com)* và đưa vào dữ liệu xác thực.

RP ID: Là định danh tên miền gốc mà máy chủ RP quản lý (ví dụ: example.com). Nếu người dùng bị lừa vào trang giả mạo (ví dụ: *exampIe.com*), trình duyệt sẽ thấy Origin không khớp với RP ID gốc, từ đó từ chối gọi thiết bị quét vân tay, giúp ngăn chặn triệt để hành vi đánh cắp thông tin.

2.3.3. Xác minh người dùng (UV) và Sự hiện diện (UP) :



Hình 2.3: Sơ đồ luồng xác thực đăng kí và xác thực đăng nhập của Webauthn.

2.3.4. Bộ đếm chữ ký (Signature Counter) chống nhân bản khóa:

Commented [KNQ4]: Em đọc thấy nội dung hình còn lỗi, được thì tách hình ra, làm rõ chữ hơn....

Mỗi khi khóa thực hiện ký số thành công, vi mạch phần cứng sẽ tự động tăng giá trị bộ đếm lên 1. Máy chủ RP sẽ lưu lại con số này. Nếu kẻ tấn công sao chép trái phép khóa phần cứng (nhân bản), thiết bị giả mạo sẽ gửi lên một bộ đếm bằng hoặc nhỏ hơn giá trị máy chủ đang lưu. Nhờ đó, máy chủ lập tức phát hiện khóa bị nhân bản và tiến hành đóng băng tài khoản.

2.4. Giao thức WebAuthn (Web Authentication API):

navigator.credentials.create(): Được kích hoạt ở luồng Đăng ký (Registration) để yêu cầu thiết bị tạo cặp khóa mới và trả về thông tin đăng ký chứa Public Key.

navigator.credentials.get(): Được kích hoạt ở luồng Đăng nhập (Authentication) để yêu cầu thiết bị sử dụng Private Key ký điện tử lên Challenge do máy chủ cung cấp.

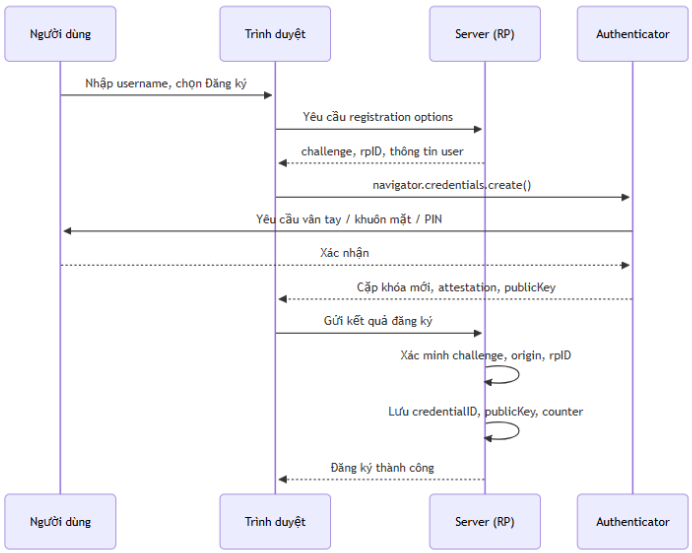
2.5. Giao thức CTAP (Client to Authenticator Protocol)

Trong khi WebAuthn kết nối Trình duyệt với Web Server, CTAP (cụ thể là CTAP2) là giao thức cho phép Trình duyệt giao tiếp với các bộ xác thực phần cứng ngoài (như khóa USB).

Do các thiết bị phần cứng có năng lực tính toán và bộ nhớ hạn chế, CTAP thực hiện việc biên dịch các đối tượng dữ liệu JSON từ môi trường web thành định dạng nhị phân ngắn gọn CBOR (Concise Binary Object Representation), giúp truyền tải dữ liệu an toàn qua các giao tiếp vật lý (USB HID, NFC, BLE).

2.6. Cơ chế hoạt động chi tiết của hai luồng cốt lõi trong hệ thống

2.6.1. Luồng Đăng ký thiết bị (Registration Ceremony)



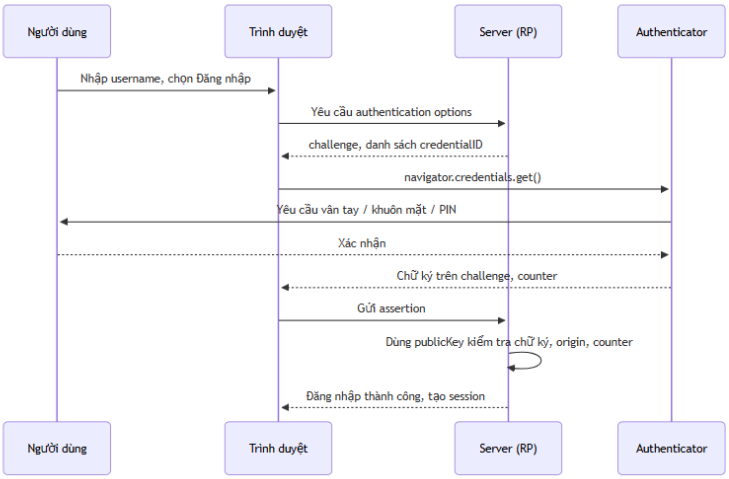
Hình 2.4: Sơ đồ chi tiết luồng đăng ký thiết bị

Khởi tạo Challenge từ Server: Khi người dùng nhập tên đăng nhập và bấm nút Đăng ký Passkey, Frontend gọi API /api/auth/register-options. Backend sử dụng thư viện @simplewebauthn/server để sinh một challenge ngẫu nhiên lưu vào Session và gửi các thông số cấu hình về Client.

Kích hoạt sinh trắc học tại Client: Trình duyệt nhận cấu hình và gọi hàm navigator.credentials.create(). Hệ điều hành hiển thị hộp thoại phần cứng (như Windows Hello hoặc Touch ID) yêu cầu người dùng quét vân tay hoặc nhập PIN.

Tạo và lưu trữ khóa: Sau khi xác thực sinh trắc học thành công, thiết bị sinh ra cặp khóa bất đối xứng. Khóa bí mật nằm lại an toàn trong chip bảo mật, trong khi Khóa công khai và Credential ID được gói ghém gửi về Backend (/api/auth/register-verify) để lưu trữ vào cơ sở dữ liệu (trong đồ án này được ánh xạ qua file db.js).

2.6.2. Luồng Xác thực đăng nhập (Authentication Ceremony):



Hình 2.5: Sơ đồ chi tiết luồng đăng nhập thiết bị.

Yêu cầu Challenge đăng nhập: Người dùng nhập tên đăng nhập và bấm Đăng nhập Passkey. Frontend gọi /api/auth/login-options, Server tạo một challenge mới cho phiên làm việc.

Ký số xác thực: Trình duyệt gọi hàm navigator.credentials.get() cùng danh sách các Credential ID đã lưu. Thiết bị yêu cầu người dùng xác nhận sinh trắc học để "mở khóa" Private Key tương ứng, sau đó ký điện tử lên tập hợp dữ liệu gồm challenge và origin (tên miền).

Xác minh chữ ký tại Server: Gói dữ liệu chữ ký số được gửi lên /api/auth/login-verify. Backend dùng Public Key tương ứng lưu trong db.js để giải mã và kiểm tra tính hợp lệ. Nếu chữ ký trùng khớp và vượt qua các tầng kiểm tra cờ trạng thái, phiên làm việc được cấp quyền và giao diện tự động chuyển hướng sang màn hình Dashboard.

2.6.3. Minh họa toán học cơ chế xác minh chữ ký ECDSA

Để làm rõ cách máy chủ (Server) có thể xác minh chính xác danh tính thiết bị mà không cần nắm giữ Khóa bí mật, dưới đây là một mô hình toán học đơn giản hóa mô phỏng thuật toán chữ ký số đường cong Elliptic (ECDSA). *(Lưu ý: Các phép toán trên đường cong Elliptic thực tế phức tạp hơn, ví dụ dưới đây sử dụng số học cơ bản để minh họa nguyên lý).*

Khởi tạo tham số hệ thống:

- ✓ **Khóa bí mật (Private Key - *d*):** Số bí mật lưu trong vi mạch của người dùng (Giả sử d = 5).

- ✓ **Điểm sinh (Base Point - *G*):** Điểm cơ sở trên đường cong được quy ước từ trước (Giả sử G = 3).
- ✓ **Khóa công khai (Public Key -K*U*):** Được tính từ Khóa bí mật và lưu trên Server.

$$KU = d \times G = 5 \times 3 = 15$$

2. Quá trình tạo chữ ký (Tại thiết bị người dùng):

- ✓ Server gửi xuống một Thử thách (***Challenge - C)*** cho lần đăng nhập (Giả sử C = 4).
- ✓ Thiết bị tự sinh một số ngẫu nhiên k cho phiên đó (Giả sử k = 7).
- ✓ Thiết bị tính toán tọa độ r trên đường cong (Giả sử kết quả r = 12).
- ✓ Thiết bị tính thành phần chữ ký s theo công thức chuẩn ECDSA:

$$s = k^{-1} \times (C + d \times r)$$

- ✓ Gói dữ liệu chữ ký (r, s) được gửi lên Server.

3. Quá trình xác minh (Tại Server):

- ✓ Server nhận được chữ ký s, tọa độ r và dùng khóa công khai KU đang lưu trong cơ sở dữ liệu để thực hiện hàm kiểm tra:

$$V = s^{-1} \times C \times G + s^{-1} \times r \times KU$$

Vì KU = d×G, khi Server thế vào và rút gọn phương trình, kết quả của phép biến đổi toán học sẽ trả về chính xác giá trị k×G.

Nếu giá trị tính ra khớp với tọa độ r ban đầu do thiết bị gửi, thuật toán kết luận Xác thực thành công.

Kết luận: Thông qua phép biến đổi đại số này, Server có thể tự tin xác nhận chữ ký s thực sự được tạo ra bởi người nắm giữ khóa bí mật d, mặc dù server hoàn toàn không biết số ngẫu nhiên k và khóa bí mật d là gì. Cơ chế này đảm bảo tính an toàn tuyệt đối cho người dùng ngay cả khi cơ sở dữ liệu của Server bị tin tặc đánh cắp.

Kết luận: Thông qua phép biến đổi đại số này, Server có thể tự tin xác nhận chữ ký *s* thực sự được tạo ra bởi người nắm giữ Khóa bí mật *d*, mặc dù Server hoàn toàn không biết số ngẫu nhiên *k* và Khóa bí mật *d* là gì. Cơ chế này đảm bảo tính an toàn tuyệt đối cho người dùng ngay cả khi cơ sở dữ liệu của Server bị tin tặc đánh cắp.

2.7. Môi trường công nghệ và các thư viện hỗ trợ lập trình:

Để hiện thực hóa hệ thống mô phỏng FIDO2/WebAuthn trong đồ án, nhóm đã lựa chọn các công nghệ nền tảng sau:

2.7.1. Ngôn ngữ và Môi trường thực thi (Node.js & Express)

Node.js: Môi trường chạy JavaScript phía máy chủ với hiệu năng cao, cung cấp khả năng xử lý bất đồng bộ tốt, phù hợp cho việc xây dựng các RESTful API xử lý logic mã hóa phức tạp.

Express.js: Framework web tối giản cho Node.js, giúp quản lý các định tuyến (Routes) như /api/auth/register-options, /api/auth/login-options và quản lý phiên làm việc bằng express-session.

2.7.2. Thư viện WebAuthn chuyên dụng

@simplewebauthn/server (Phía Backend): Thư viện mã nguồn mở tiêu chuẩn giúp đơn giản hóa việc tạo tùy chọn challenge, giải mã cú pháp CBOR, và xác minh chữ ký số WebAuthn một cách chính xác theo đặc tả của W3C.

@simplewebauthn/browser (Phía Frontend): Thư viện cung cấp các hàm bọc (wrapper) giúp gọi các API navigator.credentials.create() và navigator.credentials.get() tương thích mượt mà trên mọi trình duyệt hiện đại (Chrome, Edge, Safari).

2.7.3. Công cụ quản lý mã nguồn và phát triển

Visual Studio Code (VS Code): Trình soạn thảo mã nguồn chính của nhóm, hỗ trợ tích hợp terminal và các tiện ích mở rộng JavaScript/Node.js.

Git & GitHub: Công cụ quản lý phiên bản phân tán, hỗ trợ nhóm phân chia nhánh phát triển (feature branch), thực hiện quy trình Pull Request và kiểm tra chéo mã nguồn trước khi hợp nhất vào nhánh chính main.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể của hệ thống:

Hệ thống xác thực FIDO2 được thiết kế dựa trên mô hình kiến trúc 3 lớp (3-Tier Architecture) bám sát đặc tả của W3C, cho phép phân tách rõ ràng giữa giao diện người dùng, logic xử lý mật mã và tổ chức lưu trữ.

1. Tầng Client (Trình duyệt & Thiết bị xác thực):

- ✓ Đóng vai trò giao tiếp trực tiếp với người dùng và phần cứng. Trình duyệt web sử dụng thư viện @simplewebauthn/browser để gọi WebAuthn API.
- ✓ Tầng này trực tiếp đánh thức Bộ xác thực nội bộ (Platform Authenticator) như TPM trên Windows hoặc Secure Enclave trên macOS/iOS để thực thi thuật toán Ký số đường cong Elliptic (ECDSA).

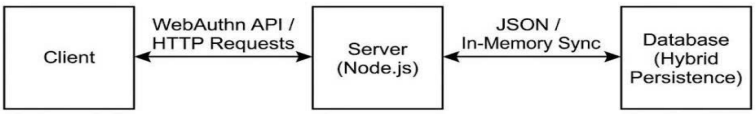
2. Tầng Server (Máy chủ Node.js):

- ✓ Hoạt động như một Relying Party (RP), được xây dựng trên nền tảng Node.js và framework Express.

- ✓ Sử dụng thư viện @simplewebauthn/server đảm nhận các tác vụ cốt lõi: Sinh chuỗi thử thách (Challenge), bóc tách dữ liệu CBOR, giải mã và xác minh chữ ký số, đồng thời quản lý phiên làm việc (express-session).

3. Tầng Database (Lưu trữ lai - Hybrid Persistence):

- ✓ Để tối ưu hóa tốc độ phản hồi tính bằng mili-giây cho các thao tác giải mã, dữ liệu người dùng được nạp và truy xuất trực tiếp trên bộ nhớ RAM (In-memory).
- ✓ Đồng thời, hệ thống sử dụng cơ chế đồng bộ hóa tự động (hàm saveUsersToFile) ghi dữ liệu xuống tệp data/users.json thông qua module fs, đảm bảo tính bền vững của khóa công khai (Public Key) khi khởi động lại máy chủ.



Hình 3.1: Sơ đồ kiến trúc tổng thể của hệ thống

3.2. Thiết kế Cơ sở dữ liệu (JSON Schema)

Do đặc thù của hệ thống xác thực Passwordless, kiến trúc dữ liệu loại bỏ hoàn toàn trường password. Dữ liệu được tổ chức theo cấu trúc NoSQL linh hoạt để lưu trữ mảng các khóa sinh trắc học và thông tin định danh sinh viên.

Bảng 3.1. Cấu trúc thực thể Người dùng (users.json)

Thuộc tính (Key)	Kiểu dữ liệu	Mô tả chi tiết chức năng
id	String	Mã định danh duy nhất (Sinh bằng Date.now() + Random). Định dạng ép kiểu sang Uint8Array để cấp cho chip bảo mật.
username	String	Tên đăng nhập của người dùng. Dùng để phân quyền (tiền tố admin hoặc student).
role	String	Vai trò tài khoản trên hệ thống.
profile	Object	Lưu trữ thông tin định danh sinh viên sau khi OCR thẻ vật lý.

- studentID	String	Mã số sinh viên (MSSV) - Khóa chính để hệ thống đối chiếu và chống mạo danh.
- isVerifiedCard	Boolean	Cờ trạng thái: Đánh dấu tài khoản đã được xác minh thẻ vật lý chính chủ hay chưa.
- fullName, dob	String	Họ tên đầy đủ và ngày sinh trích xuất từ thẻ.
credentials	**Array**	**Danh sách các Passkey đã đăng ký. Một tài khoản có thể có nhiều khóa.**
- id	String	Credential ID: Chuỗi Base64URL định danh độc nhất cho chiếc khóa vật lý.
- publicKey	Uint8Array	Khóa công khai: Lưu trữ dưới dạng mảng byte thô, dùng làm khuôn mẫu giải mã chữ ký.
- counter	Integer	Bộ đếm chữ ký (Signature Counter): Thuộc tính bắt buộc để chống nhân bản chip.

(Ghi chú chèn ảnh: Chụp màn hình file users.json hiển thị cấu trúc của 1 user đã có mảng credentials chứa publicKey dưới dạng số và chèn vào đây. Tên hình: "Hình 3.2. Cấu trúc lưu trữ dữ liệu JSON thực tế")

3.3. Sơ đồ luồng xử lý (Sequence Diagrams)

3.3.1. Luồng đăng ký sinh trắc học (Registration Flow)

Quá trình khởi tạo và ràng buộc cặp khóa bất đối xứng giữa thiết bị và máy chủ.

1. Khách hàng gửi yêu cầu đăng ký kèm username. Server kiểm tra tính duy nhất trong db.js.

2. Server gọi hàm generateRegistrationOptions, sinh ra Challenge lưu vào Session và gửi gói cấu hình chứa rpID, userID xuống Client.

3. Trình duyệt gọi navigator.credentials.create(), kích hoạt phần cứng sinh trắc học.

4. Chip bảo mật đúc cặp khóa. Khóa bí mật (Private Key) được giữ lại, Khóa công khai (Public Key) đóng gói cùng Attestation gửi lên Server.

5. Server chạy hàm verifyRegistrationResponse, nếu chữ ký hợp lệ sẽ lưu Credential ID, Public Key và khởi tạo counter = 0 vào cơ sở dữ liệu.

6. (Ghi chú chèn ảnh: Bạn sử dụng phần mềm draw.io hoặc PlantUML để vẽ Sequence Diagram cho luồng này và chèn vào. Tên hình: "Hình 3.3. Sơ đồ tuần tự luồng Đăng ký Passkey")

3.3.3. Luồng định danh và thu hồi thẻ sinh viên (Identity Reclamation)

Cơ chế bảo mật đặc thù giải quyết bài toán chống mạo danh mã số sinh viên.

1. Người dùng tải ảnh thẻ vật lý lên hệ thống. Client tiến hành trích xuất (OCR) lấy mã số sinh viên (studentId) và gửi lên API /profile/verify-card.
2. Logic hệ thống rà soát toàn bộ CSDL xem mã số sinh viên này đã tồn tại hay chưa.
3. Nếu phát hiện một tài khoản mạo danh đang chiếm giữ mã số sinh viên này (nhưng chưa qua bước xác thực thẻ vật lý gốc), hệ thống tiến hành xóa vĩnh viễn tài khoản mạo danh đó khỏi file users.json.
4. Chuyển quyền sở hữu mã số sinh viên đó cho tài khoản chính chủ đang thao tác, đồng thời bật cờ isVerifiedCard = true.

(Ghi chú chèn ảnh: Bạn chụp ảnh đoạn code xử lý thu hồi thẻ hoặc vẽ sơ đồ thuật toán và chèn vào đây. Tên hình: "Hình 3.5. Thuật toán xử lý thu hồi định danh sinh viên")

3.4. Thiết kế giao diện hệ thống

3.5. Các giải pháp an toàn thông tin áp dụng trên mã nguồn

Để chống lại các kịch bản tấn công khai thác lỗ hổng mạng phổ biến, hệ thống triển khai các chốt chặn bảo mật trực tiếp tại tầng Backend (Node.js):

Chống Tấn công giả mạo (Anti-Phishing): Hệ thống ràng buộc chặt chẽ tham số `expectedOrigin` (ví dụ: `http://localhost:3000`) và `expectedRPID` trong hàm xác minh. Mọi chữ

ký số đẩy lên từ các tên miền lừa đảo giả mạo đều bị thuật toán từ chối giải mã ngay lập tức do sai lệch ngữ cảnh.

Chống Tấn công phát lại (Replay Attack): Sử dụng Session để lưu trữ Challenge. Đoạn mã req.session.currentChallenge = null được thực thi ngay sau khi giải mã chữ ký thành công. Nếu kẻ tấn công sao chép gói tin mạng của phiên đăng nhập trước và gửi lại, hệ thống sẽ chặn đứng vì bài toán thử thách đó đã bị xóa bỏ.

Chống Nhân bản phần cứng (Hardware Clone Detection): Hệ thống liên tục giám sát sự thay đổi của biến đếm (counter). Khi phát hiện thiết bị gửi lên một counter mới nhỏ hơn hoặc bằng số liệu đang lưu trong Database, máy chủ sẽ nhận diện đây là chip đã bị sao chép (clone) trái phép và từ chối cấp quyền truy cập để bảo vệ tài khoản.

(Ghi chú chèn ảnh: Chụp ảnh đoạn code Node.js cấu hình expectedOrigin hoặc xử lý counter và chèn vào đây. Tên hình: "Hình 3.10. Cấu hình các tham số bảo mật trên Backend")

CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ THỰC NGHIỆM

4.1. Môi trường và công cụ phát triển

Để hiện thực hóa bản thiết kế từ Chương 3, hệ thống được lập trình và vận hành trên môi trường máy chủ cục bộ (localhost) với các công cụ và thông số kỹ thuật như sau:

- ✓ **Môi trường thực thi:** Node.js (phiên bản v18.x hoặc mới hơn), sử dụng trình quản lý gói npm để cài đặt thư viện.
- ✓ **Framework máy chủ (Backend):** Express.js hỗ trợ định tuyến API (Routing) và quản lý phiên làm việc (express-session).
- ✓ **Thư viện lõi FIDO2:** @simplewebauthn/server (phía Backend) và @simplewebauthn/browser (phía Frontend).
- ✓ **Công cụ soạn thảo:** Visual Studio Code (VS Code).
- ✓ **Môi trường kiểm thử (Client):** Trình duyệt Google Chrome / Microsoft Edge phiên bản mới nhất, kết hợp với Bộ xác thực nội bộ của hệ điều hành (Windows Hello hoặc Apple TouchID/FaceID).
- ✓ **Công cụ phân tích bảo mật:** Tính năng Network Tab (DevTools) và công cụ theo dõi gói tin (Live Security Inspector) để bắt và phân tích luồng dữ liệu WebAuthn.

4.2. Triển khai các chức năng cốt lõi (Backend)

4.2.1. Cấu hình Relying Party (RP) và Khởi tạo Server

Để hệ thống WebAuthn hoạt động, máy chủ phải được định danh rõ ràng. Nhóm tiến hành cấu hình hai hằng số cốt lõi làm cơ sở cho mọi quá trình mã hóa và xác minh sau này:

- ✓ rpID: 'localhost' (Định danh của Relying Party).
- ✓ expectedOrigin: 'http://localhost:3000' (Tên miền hợp lệ được phép gọi API).

(Ghi chú chèn ảnh: Chụp một đoạn code ngắn khai báo rpID và expectedOrigin trong file cấu hình/auth.js và chèn vào đây. Tên hình: "Hình 4.1. Khai báo cấu hình định danh máy chủ RP")

4.2.2. Triển khai API Luồng Đăng ký (Registration)

Quá trình đăng ký được chia thành 2 API chính:

1. **API /api/auth/register-options:** Nhận username từ Frontend, sử dụng hàm generateRegistrationOptions() của SimpleWebAuthn để tạo Challenge. Hệ thống cũng định cấu hình authenticatorSelection với userVerification: 'preferred' nhằm ưu tiên xác thực bằng sinh trắc học.
2. **API /api/auth/register-verify:** Nhận cục dữ liệu mã hóa từ thiết bị gửi lên, truyền vào hàm verifyRegistrationResponse(). Nếu verified trả về true, hệ thống sẽ trích xuất credentialID, mảng byte credentialPublicKey và counter (giá trị 0) lưu xuống users.json.

(Ghi chú chèn ảnh: Chụp đoạn code của 1 trong 2 API trên (ưu tiên hàm verify) và chèn vào đây. Tên hình: "Hình 4.2. Triển khai mã nguồn API Xác minh đăng ký thiết bị")

4.2.3. Triển khai API Luồng Đăng nhập (Authentication)

Tương tự đăng ký, đăng nhập cũng gồm 2 bước:

1. **API /api/auth/login-options:** Truy xuất CSDL để lấy mảng các credentialID của user. Dùng hàm generateAuthenticationOptions() sinh Challenge và yêu cầu thiết bị sử dụng một trong các khóa đã đăng ký.
2. **API /api/auth/login-verify:** Hàm verifyAuthenticationResponse() được gọi để dùng Public Key kiểm tra chữ ký số. Cập nhật lại counter mới vào cơ sở dữ liệu để chống nhân bản khóa, đồng thời hủy bỏ Challenge trong Session (req.session.currentChallenge = null).

(Ghi chú chèn ảnh: Chụp đoạn code cập nhật Counter hoặc đoạn set currentChallenge = null trong file js. Tên hình: "Hình 4.3. Logic kiểm tra chữ ký và cập nhật Counter khi đăng nhập")

4.2.4. Triển khai Thuật toán thu hồi định danh sinh viên (Identity Reclamation)

Trong file db.js, nhóm triển khai hàm claimStudentIdentity(userId, studentId). Thuật toán duyệt qua mảng người dùng: Nếu phát hiện một user khác đang giữ studentId này (ví dụ: 24800600488) nhưng thuộc tính isVerifiedCard là false, hệ thống sẽ dùng hàm splice() hoặc filter() để loại bỏ tài khoản mạo danh đó khỏi mảng, sau đó gán studentId cho người dùng hiện tại và cập nhật cờ thành true.

(Ghi chú chèn ảnh: Chụp đoạn code hàm xử lý xóa tài khoản mạo danh trong db.js. Tên hình: "Hình 4.4. Thuật toán thu hồi mã số sinh viên bị mạo danh")

4.3. Kiểm thử thực nghiệm và Đánh giá an toàn thông tin

Để chứng minh tính đúng đắn và độ an toàn của giao thức, nhóm tiến hành các kịch bản kiểm thử (Test Cases) bám sát các rủi ro bảo mật thực tế.

4.3.1. Kịch bản 1: Kiểm thử chức năng cơ bản (Happy Path)

- ✓ **Mục tiêu:** Đảm bảo hệ thống hoạt động trơn tru trong điều kiện bình thường.
- ✓ **Thực hiện:** Sinh viên tiến hành đăng ký tài khoản mới, sau đó đăng xuất và đăng nhập lại bằng vân tay (Windows Hello / TouchID).

Kết quả:

Trình duyệt gọi đúng popup sinh trắc học.

Sau khi quét vân tay, giao diện chuyển hướng thành công vào Dashboard.

Kiểm tra file users.json thấy khóa publicKey và credentialID đã được lưu dưới dạng mảng byte thành công (Không tồn tại mật khẩu tĩnh).

(Ghi chú chèn ảnh: Chụp màn hình console log trên terminal của Node.js hiển thị dòng chữ "Đăng nhập thành công" hoặc chụp màn hình DevTools Network trả về mã HTTP 200 OK. Tên hình: "Hình 4.5. Kết quả trả về từ Server khi xác minh sinh trắc học thành công")

4.3.2. Kịch bản 2: Kiểm thử chống tấn công lừa đảo (Anti-Phishing)

Mục tiêu: Kiểm chứng cơ chế Ràng buộc tên miền (Domain Binding) của WebAuthn.

Thực hiện: Giả lập một trang web lừa đảo bằng cách đổi cổng chạy Frontend (ví dụ đổi thành http://localhost:3001 thay vì 3000). Trình duyệt tại trang 3001 gọi API WebAuthn và gửi chữ ký số lên Server.

Kết quả:

Server phát hiện tham số Origin trong gói tin gửi lên là http://localhost:3001, không khớp với expectedOrigin (http://localhost:3000) đã khai báo ở mục 4.2.1.

Hàm verifyAuthenticationResponse văng lỗi (Throw Error).

Kết luận: Hacker không thể dụ người dùng vào trang giả mạo để lấy chữ ký đăng nhập. Hệ thống an toàn tuyệt đối trước Phishing.

(Ghi chú chèn ảnh: Chụp màn hình Terminal báo lỗi màu đỏ (Origin mismatch...) khi test thử kịch bản này. Tên hình: "Hình 4.6. Server từ chối xác thực do sai lệch tên miền (Origin)")

4.3.3. Kịch bản 3: Kiểm thử chống tấn công phát lại (Anti-Replay Attack)

Mục tiêu: Đảm bảo chữ ký số của phiên đăng nhập này không thể bị lấy cắp và dùng lại cho phiên đăng nhập sau.

Thực hiện: Sử dụng công cụ theo dõi gói tin (Live Security Inspector / Network Tab) để chép lại toàn bộ cục dữ liệu JSON (chứa chữ ký số) mà thiết bị vừa gửi lên Server lúc đăng nhập thành công. Sau đó, dùng Postman (hoặc fetch) gửi lại y hệt cục dữ liệu đó lên API /api/auth/login-verify một lần nữa.

Kết quả:

Server từ chối gói tin ngay lập tức vì chuỗi Challenge đính kèm trong chữ ký đó không còn khớp với Challenge hiện tại trong Session (do đã bị reset về null sau lần đăng nhập đầu tiên).

Kết luận: Hệ thống vô hiệu hóa hoàn toàn phương thức tấn công Replay Attack.

(Ghi chú chèn ảnh: Chụp màn hình báo lỗi "Challenge mismatch" hoặc "Missing challenge" trên Terminal/Postman. Tên hình: "Hình 4.7. Chặn đứng hành vi gửi lại gói tin cũ (Replay Attack)")

4.3.4. Kịch bản 4: Kiểm thử chống nhân bản thiết bị (Hardware Clone Detection)

Mục tiêu: Phát hiện hành vi sao chép trái phép chip bảo mật.

Thực hiện: Can thiệp thủ công vào file users.json, cố tình chỉnh sửa tham số counter của một người dùng lên mức 100. Sau đó tiến hành dùng thiết bị thật quét vân tay đăng nhập. Thiết bị thật gửi lên chữ ký kèm counter bằng 10 (nhỏ hơn 100).

Kết quả:

Hàm kiểm tra của SimpleWebAuthn phát hiện counter gửi lên (10) ≤ counter trong DB (100).

Server nhận diện đây là khóa bị nhân bản (Clone) và báo lỗi, từ chối cấp Session đăng nhập.

Kết luận: Thuộc tính counter hoạt động chính xác, giúp hệ thống phát hiện và cô lập các chìa khóa vật lý bị làm giả.

-

(Ghi chú chèn ảnh: Chụp đoạn báo lỗi liên quan đến "Counter" trên màn hình console. Tên hình: "Hình 4.8. Server phát hiện và từ chối thiết bị có bộ đếm bất thường")

4.4. Tổng kết chương

Thông qua việc xây dựng và cấu hình mã nguồn Backend Node.js, kết hợp với các công cụ kiểm thử thực nghiệm, nhóm đã triển khai thành công một hệ thống xác thực hoàn toàn không sử dụng mật khẩu. Kết quả kiểm thử (Test Cases) ở Mục 4.3 chứng minh hệ thống không chỉ hoạt động ổn định về mặt chức năng (Đăng ký/Đăng nhập), mà còn đáp ứng xuất sắc các tiêu chuẩn bảo mật khắt khe nhất của FIDO2, chống lại thành công các kịch bản tấn công nguy hiểm như Phishing, Replay Attack và Hardware Cloning.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Tổng kết những kết quả đạt được

Trải qua quá trình nghiên cứu lý thuyết chuyên sâu và tiến hành thực nghiệm, nhóm đồ án đã hoàn thành xuất sắc các mục tiêu đề ra ban đầu, giải quyết trọn vẹn bài toán xác thực không mật khẩu (Passwordless). Cụ thể, đề tài đã đạt được những dấu ấn quan trọng sau:

Về mặt lý thuyết: Đề tài đã bóc tách thành công cơ chế hoạt động phức tạp của tiêu chuẩn FIDO2 và giao thức WebAuthn. Nhóm đã làm rõ được cách thức hệ mật mã khóa công khai (Public-Key Cryptography) kết hợp với các giao thức CTAP để tạo ra một hàng rào bảo mật vững chắc, loại bỏ hoàn toàn sự phụ thuộc vào mật khẩu truyền thống.

Về mặt thực tiễn (Triển khai hệ thống):

- ✓ Xây dựng thành công máy chủ Relying Party (RP) bằng Node.js và Express, tích hợp chuẩn xác bộ thư viện @simplewebauthn.

- ✓ Thiết kế giao diện Frontend tối giản, thân thiện, kết nối mượt mà với API sinh trắc học nguyên bản của hệ điều hành (Windows Hello, Touch ID/Face ID).
- ✓ Cấu hình thành công kiến trúc cơ sở dữ liệu phi mật khẩu (chỉ lưu trữ Public Key và Credential ID).

Về mặt an toàn thông tin: Thông qua các kịch bản kiểm thử ở Chương 4, hệ thống đã chứng minh được khả năng phòng ngự tuyệt đối trước các cuộc tấn công mạng nguy hiểm nhất hiện nay như lừa đảo giả mạo tên miền (Phishing), tấn công phát lại (Replay Attack) và sao chép phần cứng (Hardware Cloning).

Tính năng đột phá: Đề tài đã tích hợp thành công thuật toán "Thu hồi định danh sinh viên", giúp giải quyết triệt để bài toán mạo danh mã số sinh viên trên môi trường số.

5.2. Những mặt hạn chế của đồ án

Dù đã mô phỏng thành công lõi xác thực của FIDO2, đồ án vẫn còn một số giới hạn nhất định do rào cản về mặt thời gian và cơ sở hạ tầng thiết bị:

Giới hạn về cơ sở dữ liệu: Để phục vụ mục tiêu chứng minh khái niệm (Proof of Concept) và tối ưu hóa thời gian phản hồi, hệ thống hiện đang lưu trữ dữ liệu người dùng dưới dạng mảng JSON trên bộ nhớ RAM (In-memory). Cấu trúc này chưa phù hợp để lưu trữ lượng dữ liệu khổng lồ hàng triệu người dùng trong thực tế.

Giới hạn về môi trường triển khai: Ứng dụng hiện chỉ đang được cấu hình để chạy thử nghiệm trên môi trường máy chủ cục bộ (localhost).

Chưa có cơ chế khôi phục tài khoản (Account Recovery): Khóa bí mật (Private Key) được gắn chết vào phần cứng vật lý của người dùng. Trong phiên bản hiện tại, nếu người dùng làm mất thiết bị hoặc cài lại hệ điều hành, họ sẽ mất vĩnh viễn quyền truy cập vào tài khoản do hệ thống chưa phát triển tính năng cấp lại quyền truy cập khẩn cấp.

5.3. Định hướng phát triển tương lai

Từ những hạn chế trên, nhóm đề xuất các hướng phát triển tiếp theo để nâng cấp đề tài thành một sản phẩm phần mềm thương mại hoàn chỉnh:

Nâng cấp Hệ quản trị Cơ sở dữ liệu: Chuyển đổi từ mô hình lưu trữ file JSON sang các hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ như PostgreSQL, SQL Server hoặc cơ sở dữ liệu NoSQL như MongoDB để tăng tính toàn vẹn và khả năng mở rộng (Scale).

Triển khai lên môi trường thực tế (Production): Đưa máy chủ Node.js lên các nền tảng đám mây (Cloud Hosting) như AWS, Heroku hoặc Vercel, đồng thời cấu hình chứng chỉ bảo mật HTTPS/SSL (điều kiện bắt buộc để trình duyệt cho phép chạy WebAuthn trên internet).

Xây dựng tính năng Khôi phục tài khoản (Recovery Flow): Tích hợp thêm các phương thức dự phòng như gửi Magic Link qua Email hoặc mã OTP qua SMS. Điều này giúp người dùng dễ dàng khôi phục quyền truy cập khi thiết bị sinh trắc học bị hỏng hoặc thất lạc, đồng thời có thể xóa (Revoke) khóa bảo mật cũ ra khỏi hệ thống.

Mở rộng hỗ trợ đa thiết bị (Cross-device Authentication): Nghiên cứu và áp dụng chuẩn giao tiếp Bluetooth/QR Code của FIDO2 (Hybrid Transport) để người dùng có thể sử dụng điện thoại thông minh quét mã QR trên màn hình máy tính bàn (không có vân tay) để xác thực đăng nhập từ xa.